

Comparative Evaluation of Decision Tree and XGBoost Regression on Financial and Wage Prediction Tasks

Mushtasin¹

¹Department of Computer Science and Engineering, North South University

August 5, 2026

Abstract

This report presents a comparative empirical study of two supervised regression algorithms, Decision Tree Regression and Extreme Gradient Boosting (XGBoost) Regression, applied to two independent real-world tabular datasets: a credit card balance dataset (RG-Credit, $n = 400$) and a wage prediction dataset (RG-Wage, $n = 3000$). Both datasets are split *sequentially* (without shuffling) into training (70%), validation (15%), and test (15%) subsets. Hyperparameters for both models are selected exclusively using the validation subset, and a manually implemented (non-library) Mean Squared Error (MSE) function is used for all reported evaluation metrics. On the RG-Credit dataset, XGBoost substantially outperforms the Decision Tree on the held-out test set (test MSE of 3,949.20 versus 19,674.27). On the RG-Wage dataset, the two models perform much closer to one another, with XGBoost again achieving the lower test MSE (1,228.44 versus 1,252.94). We discuss these results in terms of dataset size, feature-target relationship complexity, and the bias-variance behaviour of tree ensembles relative to single decision trees.

1 Introduction

Regression tasks require models that can capture the functional relationship between a set of predictor variables and a continuous target variable. Tree-based models are a natural choice for tabular data because they require little preprocessing, handle mixed numerical and categorical predictors gracefully, and can model non-linear interactions automatically. Two widely used tree-based regressors are (i) the single **Decision Tree Regressor**, which partitions the feature space through a sequence of axis-aligned splits, and (ii) **XGBoost**, a gradient-boosted ensemble of shallow decision trees trained additively to minimize a regularized objective.

The purpose of this assignment is threefold: (1) to gain hands-on experience with Decision Tree and XGBoost regression, (2) to practice principled hyperparameter selection using an explicit validation split, and (3) to demonstrate a thorough understanding of regression evaluation, in particular by implementing the Mean Squared Error metric manually rather than relying on library functions. The complete pipeline — data loading, sequential splitting, model training, hyperparameter search, manual evaluation, and results comparison — is applied independently to two datasets that differ substantially in size, feature composition, and target behaviour, allowing the discussion to generalize beyond a single case.

2 Dataset Description

2.1 RG-Credit Dataset

The RG-Credit dataset contains 400 observations describing customers of a credit card issuer. The prediction target is **Balance**, the customer’s average credit card balance. The predictors are:

- **Numerical:** **Income**, **Limit**, **Rating**, **Cards**, **Age**, **Education**.
- **Categorical:** **Own** (home ownership), **Student**, **Married**, **Region**.

No missing values are present in any column. Categorical variables are one-hot encoded (with the first category dropped to avoid redundancy), producing 11 model-ready features.

2.2 RG-Wage Dataset

The RG-Wage dataset contains 3,000 observations of male workers, with **wage** as the prediction target. The raw file also contains a **logwage** column, which is a deterministic transformation ($\log(\text{wage})$) of the target itself; this column is **dropped** prior to modelling because retaining it would constitute direct target leakage. The **region** column takes a single constant value across all 3,000 rows and therefore carries no predictive information; it is also dropped. The remaining predictors used are:

- **Numerical:** **year**, **age**.
- **Categorical:** **maritl** (marital status), **race**, **education**, **jobclass**, **health**, **health_ins**.

No missing values are present. One-hot encoding of the categorical predictors yields 16 model-ready features.

3 Methodology

3.1 Sequential Data Splitting

Per the assignment requirements, no random shuffling is performed. For each dataset, the rows are split in their original order into:

- Training set: first 70% of rows,
- Validation set: next 15% of rows,
- Test set: final 15% of rows.

This was implemented manually with NumPy index arithmetic (`floor(0.70n)` and `floor(0.85n)` as the split boundaries) rather than with `sklearn.model_selection.train_test_split`, since that utility shuffles by default and cannot guarantee an exact, order-preserving three-way split. Table 1 shows the resulting subset sizes.

Table 1: Sequential split sizes for both datasets.

Dataset	Train	Validation	Test
RG-Credit	280	60	60
RG-Wage	2100	450	450

3.2 Manual Mean Squared Error

As required, no library metric function (e.g. `sklearn.metrics.mean_squared_error`) is used anywhere in the pipeline. MSE is implemented directly with NumPy according to its standard definition:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

where y_i is the true target value, $\hat{y}_i$ is the model's predicted value, and n is the number of samples. The implementation proceeds in four explicit steps: (1) compute the residual $y_i - \hat{y}_i$ for every sample, (2) square each residual, (3) sum the squared residuals across all n samples, and (4) divide the sum by n to obtain the mean. This function is the *only* scoring mechanism used for hyperparameter selection (on the validation set) and for final reporting (on the test set), for both models and both datasets.

3.3 Decision Tree Regressor

`sklearn.tree.DecisionTreeRegressor` was used. The following hyperparameters were tuned:

- **max_depth** $\in \{3, 5, 7, 10, \text{None}\}$ — controls the maximum depth of the tree; shallow trees underfit, unconstrained trees overfit.
- **min_samples_split** $\in \{2, 5, 10, 20\}$ — minimum number of samples required to split an internal node; larger values produce coarser, more regularized trees.
- **min_samples_leaf** $\in \{1, 2, 5, 10\}$ — minimum number of samples required at a leaf; larger values smooth predictions and reduce variance.
- **criterion** $\in \{\text{squared_error}, \text{friedman_mse}\}$ — the impurity/split-quality function used to choose splits.

These parameters were chosen because they directly govern the depth/complexity and leaf-granularity of the tree, which are the primary levers for controlling the bias-variance trade-off of a single decision tree. An exhaustive grid search over all $5 \times 4 \times 4 \times 2 = 160$ combinations was performed for each dataset; every combination was trained on the training set and scored on the validation set using the manual MSE function, and the combination with the lowest validation MSE was retained.

3.4 XGBoost Regressor

`xgboost.XGBRegressor` (objective `reg:squarederror`) was used. The following hyperparameters were tuned:

- **max_depth** $\in \{3, 5, 7\}$ — depth of each individual boosted tree.
- **learning_rate** $\in \{0.01, 0.05, 0.1\}$ — shrinkage applied to each boosting step.
- **n_estimators** $\in \{100, 300\}$ — number of boosting rounds (trees).
- **subsample** $\in \{0.8, 1.0\}$ — fraction of training rows sampled per tree.
- **colsample_bytree** $\in \{0.8, 1.0\}$ — fraction of features sampled per tree.
- **min_child_weight** $\in \{1, 5\}$ — minimum sum of instance weight needed in a child node.

- **gamma** $\in \{0, 0.1\}$ — minimum loss reduction required to make a further split.

These parameters jointly control (a) the complexity of each base learner (**max_depth**, **min_child_weight**, **gamma**), (b) the pace and stability of the boosting process (**learning_rate**, **n_estimators**), and (c) the amount of stochastic regularization injected via row/column subsampling (**subsample**, **colsample_bytree**). The full Cartesian product of this grid contains $3 \times 3 \times 2 \times 2 \times 2 \times 2 \times 2 = 288$ combinations. Exhaustively evaluating all 288 combinations for both datasets is computationally expensive without a proportionate benefit, so a **fixed-seed random search** over 60 unique combinations drawn from the same grid was used instead — a standard and well justified practical alternative to full grid search for gradient boosting models, while still exercising every hyperparameter listed in the assignment. Each sampled combination was trained on the training set and scored on the validation set with the manual MSE function; the combination with the lowest validation MSE was retained as the final model configuration.

3.5 Final Retraining and Test Evaluation

For each model and dataset, once the best hyperparameter combination was identified using only the validation set, that exact configuration (already fit on the training set during the search) constitutes the final model. Consistent with the assignment instructions, the validation set is used strictly for model *selection* and never for parameter *fitting*; the final, selected model is evaluated exactly once on the previously untouched test set, again using the manual MSE implementation of Equation 1.

4 Experimental Setup

All experiments were implemented in Python using **pandas** and **NumPy** for data handling, **scikit-learn** for the Decision Tree Regressor, and the official **xgboost** package for the XGBoost Regressor. A fixed random seed (**random_state=42**) was used throughout for reproducibility of model initialization and the XGBoost random search sampling. No evaluation-metric library function was used at any point; Equation 1 was implemented directly with NumPy array operations. The full pipeline was executed independently and identically (same code path) for both the RG-Credit and RG-Wage datasets.

5 Hyperparameter Selection

Table 2 reports the best hyperparameter combination found for each model/dataset pair, selected purely by minimizing validation MSE.

Table 2: Best hyperparameters selected on the validation set.

Dataset	Model	Hyperparameters	Validation MSE
RG-Credit	Decision Tree	max_depth=10, min_samples_split=2, min_samples_leaf=2, criterion=squared_error	14,858.32
RG-Credit	XGBoost	max_depth=7, learning_rate=0.05, n_estimators=300, subsample=0.8, colsample_bytree=1.0, min_child_weight=1, gamma=0.1	7,694.60
RG-Wage	Decision Tree	max_depth=5, min_samples_split=2, min_samples_leaf=10, criterion=squared_error	1,323.67
RG-Wage	XGBoost	max_depth=3, learning_rate=0.05, n_estimators=300, subsample=0.8, colsample_bytree=1.0, min_child_weight=1, gamma=0.1	1,226.18

For RG-Credit, the Decision Tree favours a relatively deep tree (depth 10) with minimal leaf/split constraints, indicating that the validation set rewards a fine-grained partition of the small feature space. XGBoost, in contrast, selects a moderate depth (7) combined with a low learning

rate (0.05), a large ensemble (300 trees), row subsampling (0.8), and a small positive `gamma`, reflecting the value of regularized, incremental boosting over a single unconstrained tree on this dataset. For RG-Wage, the Decision Tree prefers a much shallower tree (depth 5) with a larger minimum leaf size (10), suggesting the signal-to-noise ratio in wage data is lower and heavy pruning is beneficial; XGBoost again selects a low learning rate (0.05) and 300 trees, but a shallower per-tree depth (3), consistent with wage data being noisier and better modelled by an ensemble of weak learners than by a single tree of any depth.

6 Results

6.1 RG-Credit Dataset

Table 3: RG-Credit: validation and test MSE for the selected models.

Model	Validation MSE	Test MSE
Decision Tree	14,858.32	19,674.27
XGBoost	7,694.60	3,949.20

6.2 RG-Wage Dataset

Table 4: RG-Wage: validation and test MSE for the selected models.

Model	Validation MSE	Test MSE
Decision Tree	1,323.67	1,252.94
XGBoost	1,226.18	1,228.44

6.3 Overall Comparison

Table 5: Decision Tree vs. XGBoost: test-set MSE comparison across both datasets.

Dataset	Decision Tree (Test MSE)	XGBoost (Test MSE)	Relative Improvement
RG-Credit	19,674.27	3,949.20	$\approx 79.9\%$ reduction
RG-Wage	1,252.94	1,228.44	$\approx 2.0\%$ reduction

On both datasets XGBoost achieves a lower test MSE than the single Decision Tree; the magnitude of the improvement, however, differs substantially between the two tasks.

7 Discussion

RG-Credit. The improvement from XGBoost over the Decision Tree is large (roughly a $4\times$ reduction in test MSE). With only 280 training rows and 11 features, a single decision tree with `max_depth=10` is prone to overfitting: it can create very specific, low-support splits that memorize idiosyncrasies of the small training set, which explains why its validation MSE (14,858) is already noticeably worse than XGBoost’s (7,695), and why the gap widens further on the test set (19,674 vs.

3,949). XGBoost’s ensemble of many shallow, regularized trees, combined with a low learning rate and row subsampling, averages out this variance and generalizes considerably better on a dataset of this size.

RG-Wage. Here the two models perform much more similarly (test MSE 1,252.94 vs. 1,228.44, a difference of about 2%). With 2,100 training rows, a single decision tree constrained to depth 5 is far less prone to overfitting than the deep tree used for RG-Credit, and the underlying wage-vs-demographics relationship in this dataset is comparatively noisy and only weakly non-linear given the available predictors (age, year, and categorical demographic variables), which limits how much additional structure any tree-based model — ensemble or not — can extract. This is consistent with the well-known result that boosting yields its largest gains over a single tree when (a) the base tree is prone to high variance and (b) enough data and non-linear signal exist for an ensemble to exploit; RG-Wage satisfies neither condition as strongly as RG-Credit.

General observations. Across both datasets, the best XGBoost configuration consistently favoured a low learning rate (0.05) with a relatively large number of estimators (300), reflecting the standard boosting trade-off in which many small, conservative updates outperform few large ones. The Decision Tree’s optimal depth adapted sensibly to each dataset’s noise level (deeper for the cleaner, smaller RG-Credit task; shallower for the noisier, larger RG-Wage task), which itself is evidence that the validation-based hyperparameter search behaved as intended.

8 Conclusion

This study implemented and compared Decision Tree Regression and XGBoost Regression on two structurally different real-world regression datasets, using a strict sequential train/validation/test protocol, validation-only hyperparameter selection, and a fully manual implementation of the Mean Squared Error metric. XGBoost outperformed the single Decision Tree on both datasets, with a very large margin on the smaller, less noisy RG-Credit dataset and a modest margin on the larger, noisier RG-Wage dataset. These results are consistent with the theoretical expectation that gradient-boosted ensembles reduce the variance of individual decision trees, with the size of that benefit depending on how overfitting-prone the single-tree baseline is for the task at hand. Future work could extend this comparison with Random Forests as an additional variance-reduction baseline, feature importance analysis, and residual diagnostics to further characterize each model’s error structure.

Appendix: Reproducibility Notes

All code was executed with a fixed random seed (`random_state=42`). The complete, commented Python implementation (data loading, sequential splitting, Decision Tree and XGBoost training and tuning, manual MSE computation, and results aggregation) is provided as a companion file, `regression_pipeline.py`, organized into the eleven sections specified by the assignment: (1) imports, (2) dataset loading, (3) data inspection, (4) sequential split, (5) Decision Tree implementation, (6) Decision Tree hyperparameter tuning, (7) XGBoost implementation, (8) XGBoost hyperparameter tuning, (9) manual MSE implementation, (10) final evaluation, and (11) results comparison.